Unidad 4 / Escenario 8

Lectura fundamental

Contenedores y tipos de datos definidos por el usuario

Contenido

1	Asociaciones	1
2	Problema de aplicación	1
3	Clases en el ejemplo tienda	2
4	Problemas propuestos	10

Palabras clave: atributos, constructores, *getters*, método *toString*.

1. Asociaciones

Las principales asociaciones que se pueden usar e implementar en lenguajes de programación de alto nivel de propósito general son las agregaciones y composiciones.

La composición es una relación entre objetos que se puede entender como: *X* está compuesto de uno (o varios) *Y*. Por ejemplo, una silla tiene un espaldar determinado. La agregación, a diferencia de la composición, se debe entender como una relación de tipo *X* tiene uno (o varios) *Y*. Por ejemplo, un automóvil tiene varios pasajeros o un automóvil tiene cierto remolque. La diferencia puede ser un poco confusa e, incluso, a veces subjetiva o relativa al problema que se esté modelando. Esta similitud se acentúa en cuanto a que en la implementación de ambas relaciones se pueden usar estrategias y estructuras muy similares.

Se puede diferenciar, entonces, la agregación de la composición por la posibilidad de los objetos de existir o tener sentido, independientemente del objeto grande o contenedor en el problema determinado que se esté resolviendo.

2. Problema de aplicación

Analice el siguiente ejemplo escogido es una abstracción bastante simple y limitada de lo que es una tienda, en el que se requieren los siguientes datos: nombre de la tienda, clientes (nombre e identificación), ítems de la *tienda* (nombre, precio y cantidad de elementos disponibles en inventario) y facturas. Cada factura debe tener asociado un cliente, así como conjunto de ítems, cada uno con su cantidad.

3. Clases en el ejemplo tienda

En esta sección se explican y muestran las clases usadas para el ejemplo de la tienda. Estas sirven como punto de partida para los ejercicios de aplicación en la sección de problemas propuestos.

Usted debe notar que algunas clases tienen aspectos en común. En particular, las clases *Store*, *Bill*, *Client* e *Item* son todas abstracciones de objetos del mundo real; es decir, son versiones simplificadas de lo que son cada uno de esos tipos de elementos. Es por esto que en el ámbito de la programación orientada por objetos se les conoce como clases entidad. Todas ellas tienen aspectos que comparten, a saber:

1. **Atributos:** todas las clases tienen un atributo nombre. *Bill*, *Client* e *Item* tienen un atributo *id*.

2. **Constructores**: son los métodos que tienen el mismo nombre de la clase.

3. ***Getters***: al igual que en el Escenario anterior, las clases entidad tienen varios métodos *getters* que sirven para recuperar valores de atributos desde fuera de la clase.

4. **Método *toString***: como se vio en el Escenario anterior, existe la posibilidad de sobrescribir métodos en Java. En particular, la sobrescritura del método *toString* se usa en este ejemplo para facilitar la impresión a la salida estándar de la información de las instancias de las clases entidad.

5. **Implementaciones de la interfaz comparable**: las interfaces son un aspecto importante en el lenguaje de programación Java. De manera estándar, hay muchas interfaces implementadas, pero, dado el alcance del Módulo, no se puede profundizar en este tema. Para más detalles, se recomienda ver referencias del lenguaje de programación Java, como Sierra y Bates (2005). En este ejemplo particular, el uso de la interfaz Comparable hace que las diferentes instancias de una clase se puedan comparar bajo algún criterio particular. El criterio escogido en los casos de *Clients*, *Bills* e *Items* es el *id*. Para conocer más sobre implementación de interfaces en Java, ver la documentación recomendada de Oracle (1999a). Para este ejemplo, basta con entender que el hecho de implementar la interfaz, como se hace, fuerza al programador a implementar el método *compareTo*, donde debe estar la lógica necesaria para determinar si el objeto que llama al método va antes (retorna -1), después (retorna 1) o es considerado igual (retorna 0) a la instancia que recibe como argumento. Por ello, las clases *Item*, *Bill* y *Client* tienen implementada la interfaz *Comparable*, lo cual se hace seguido de la declaración de la clase al inicio del archivo. Estas tres clases tienen también un método *compareTo* implementado, como se describió anteriormente.

3.1. *Main*

Esta clase contiene el método *main* donde inicia el programa. En este método se instancia la clase *Store*, creando un ejemplar de tienda. Seguido a esto se crean y agregan a la tienda dos clientes: Bob y Ann. Después, se establecen cinco ítems y se agregan en ciertas cantidades al inventario de la tienda: 10 arepas, 20 huevos, 15 churros, 30 arroz, 5 panes.

Después, se hacen dos impresiones a la salida estándar. En la primera se muestra la tienda completa; es decir, se muestra lo que retorna el método ***toString*** de la clase *Store*. El segundo exhibe lo que retorna el método ***getInventory*** de la clase *Store*, la cual es una representación del inventario mostrado ordenado por orden alfabético respecto a los nombres de los productos. Este ordenamiento se hace dentro del método, mediante un llamado al método *sort* de la clase *Collections*, con una instancia de un comparador por nombre, que es una instancia de la clase *ByNameItemComparator*.

Finalmente, se instancia la clase Factura (*Bill*), creando una factura para el cliente Bob con 2 arepas y 4 huevos. Se hace la impresión a la salida estándar de la factura, la cual se crea en el método ***toString*** de la clase *Bill*.

```java
public class Main {
    public static void main(String[] args) {
        Store store = new Store("My store");
        Client ann = new Client("Ann", 100);
        Client bob = new Client("Bob", 200);
       store.addClient(ann);
       store.addClient(bob);
       Item arepa = new Item(1,"Arepa",1000);
       Item egg = new Item(2,"Egg",200);
       Item churro = new Item(3,"Churro",400);
       Item rice = new Item(4,"Rice",2000);
       Item bread = new Item(5,"Bread",100);
       store.addItem(arepa, 10);
       store.addItem(egg, 20);
       store.addItem(churro, 15);
       store.addItem(rice, 30);
       store.addItem(bread, 5);
       System.out.println(store);
       System.out.println("Inventory: "+store.getInventory());
       Bill bill = new Bill(1,bob);
       bill.addItem(arepa, 2);
       bill.addItem(egg, 4);
       System.out.println(bill);
    }
}
```

Código 1: Clase *Main* en el ejemplo de la Tienda

3.2. *ByNameItemComparator*

La clase *ByNameItemComparator* es una clase corta que se usa en el ejemplo de la tienda y que ofrece la posibilidad de mostrar los ítems en el inventario, pero no ordenados por *id*, como está por defecto, sino ordenados por nombre. Esta clase implementa la interfaz Comparator, la cual, dado que se implementó con el argumento de la clase *Item* (*implements Comparator¡Item¿*), debe implementar el método compare, que recibe dos instancias de la clase Item. Este método debe tener la lógica necesaria para decidir si el primer argumento es menor (retorna -1), mayor (retorna 1) o igual (retorna 0) al segundo argumento. Esta clase se usa únicamente en el método ***getInventory*** de la clase *Store*.

```java
import java.util.Comparator;
public class ByNameItemComparator
    implements Comparator<Item> {
    @Override
    public int compare(Item a, Item b) {
        return a.getName().compareToIgnoreCase(b.getName());
    }
}
```

Código 2: Clase *ByNameItemComparator* en el ejemplo de la Tienda

3.3. *Client*

Esta clase presenta una abstracción bastante simplificada de lo que sería un cliente para el ejemplo de la tienda: tiene un *id* y un nombre. Tiene un constructor que inicializa una instancia de Cliente con un *id* y nombre que recibe por parámetro, al igual que un par de métodos ***getters***, el método ***toString*** e implementa la interfaz *Comparable*, lo cual obliga a que se implemente el método ***compareTo***. Implementar la interfaz *Comparable* es necesario para que las instancias de *Client* se puedan almacenar en un contenedor tipo *TreeSet*, como el que tiene la clase *Store*.

```java
public class Client implements Comparable<Client> {
        private String name;
        private long id;
        public Client(String name, long id) {
            this.name = name;
            this.id = id;
        }
        public String getName() {
            return name;
        }
        public long getId() {
            return id;
        }
        @Override
        public String toString() {
            return this.id+"\t"+this.name;
        }
        @Override
        public int compareTo(Client other) {
         if( this.getId() < other.getId() )
             return -1;
         if( this.getId() > other.getId() )
             return 1;
         return 0;
    }
}
```

Código 3: Clase *Client* en el ejemplo de la Tienda

3.4 *Item*

La clase *Item* es muy similar a la clase *Client*, salvo que esta tiene, adicionalmente, un atributo pricePerUnit, el cual almacena el valor de un ejemplar de una instancia dada.

```java
public class Item implements Comparable<Item>{
    private long id;
    private String name;
    private int pricePerUnit;
    public Item(long id, String name, int pricePerUnit) {
        this.id = id;
        this.name = name;
        this.pricePerUnit = pricePerUnit;
      }
      @Override
      public String toString() {
          return id + "\t" + name + "\t";
      }
      public long getId() {
          return id;
      }
      public String getName() {
          return name;
      }
      public int getPricePerUnit() {
          return pricePerUnit;
      }
      @Override
      public int compareTo(Item other) {
         if( this.getId() < other.getId() )
             return -1;
         if( this.getId() > other.getId() )
             return 1;
         return 0;
     }
 }
```

Código 4: Clase *Item* en el ejemplo de la Tienda

3.5. *Bill*

Esta clase es una abstracción de lo que sería una factura. Tiene como atributo un cliente, es decir, que hay una asociación entre factura y cliente, puesto que, una factura debe asignarse a un cliente. Adicionalmente, posee un diccionario en donde las llaves serán instancias de la clase *Item* y los valores serán enteros; estos enteros representan la cantidad de ejemplares de un ítem dado que se vendieron y se registraron en la factura. Así, si por ejemplo, un cliente compra 10 huevos, en el diccionario ítems estará la pareja (huevo,10). Esta clase también implementa la interfaz *Comparable*, tiene sobrescrito el método ***toString*** y tiene algunos métodos ***getters***. El análisis detallado de sus métodos se deja como ejercicio, el cual debe hacerse como se ha recomendado siempre en el Módulo, con un seguimiento y análisis paso a paso del programa.

```java
import java.util.TreeMap;
public class Bill implements Comparable<Item>{
    private int id;
    private Client client;
    private TreeMap<Item,Integer> items;
    public Bill(int id, Client client) {
        this.id = id;
        this.client = client;
        this.items = new TreeMap<Item,Integer>();
    }
    public void addItem( Item item, int ammount )
    {
        if( items.containsKey( item ) )
            this.items.put(item , this.items.get(item)+ammount);
        else
            this.items.put(item, ammount);
    }
    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("BILL: "+client.getName()+" ID: "+client.getId()+"\n");
        for( Item i : this.items.keySet() )
            sb.append(i.getName()+"\t"+this.items.get(i)+" x "+i.getPricePerUnit()+"\n");
        sb.append("Total\t"+this.getBillCost()+"\n");
        return sb.toString();
    }
    public int getBillCost(){
        int ans = 0;
        for( Item i : items.keySet() )
            ans += this.items.get(i)*i.getPricePerUnit();
        return ans;
    }
    @Override
    public int compareTo(Item other){
        if( this.getId() < other.getId() )
            return -1;
        if( this.getId() > other.getId() )
            return 1;
        return 0;
    }
    private int getId() {
        return this.id;
    }
}
```

Código 5: Clase *Bill* en el ejemplo de la Tienda

3.6. *Store*

Finalmente, la clase más grande del programa es *Store*. Esta clase tiene toda la información de la tienda. Cada instancia de la tienda (en el programa se espera se instancie solo una) tiene, entonces, los siguientes atributos y comportamientos:

1. ***name***: un nombre, que será un *String*. Es el único valor que recibe el constructor para inicializar una instancia de la clase *Store*.
2. ***TreeMap ¡Item,Integer¿ inventory***: este diccionario almacenará el inventario actual de la tienda; así, entonces, habrá una pareja (elemento en inventario, cantidad) por cada elemento ingresado al inventario.
3. ***TreeSet¡Bill¿ bills:*** este contenedor almacenará todas las facturas creadas. No habrá dos facturas con el mismo *id*; este problema se resuelve mediante la implementación propia del *TreeSet* usado como contenedor.
4. ***TreeSet¡Client¿ clients:*** este contenedor almacenará la información de los clientes; de igual manera que con los *Bill*, no guarda dos clientes con el mismo *id*.
5. ***void addItem (Item item, int ammount)***: este método agregará cierta cantidad de elementos de un ítem dado. Así, por ejemplo, si se hace el llamado *addItem* (egg, 10), se adicionarán 10 huevos al inventario; si los huevos aún no están en el conjunto de llaves del contenedor, este método lo agregara´ con un valor inicial asociado de 10.
6. ***addClient(Client c)***: este método añade un cliente al contenedor de clientes (*clients)*.
7. ***addBill (Bill b)***: este método agrega una instancia de *Bill* al contenedor de facturas *bills*.
8. ***String getInventory():*** este método retorna una representación en cadena del inventario. La representación tiene los elementos de inventarios organizados por nombre. Esta organización, que no es la que tienen por defecto los elementos en el diccionario *inventory*, se logra mediante el uso de una instancia de la clase ByN *ameItemComparator* y el método ***sort*** de la clase *Collections.*

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.TreeMap;
import java.util.TreeSet;
public class Store {
    private String name;
    private TreeMap    <Item,Integer> inventory;
    private TreeSet<Bill> bills;
    private TreeSet<Client> clients;
    public Store(String storeName) {
        this.name = storeName;
        this.inventory = new TreeMap<Item,Integer>();
        this.bills = new TreeSet<Bill>();
        this.clients = new TreeSet<Client>();
    }
    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append(this.name+"\n");
        sb.append(this.clients.size()+" Clients: \n");
        sb.append("ID\tName\n");
        for( Client c : this.clients)
            sb.append(c+"\n");
        sb.append("Inventory:\n");
        sb.append("ID\tName\t\tStock\n");
        for( Item k : inventory.keySet())
            sb.append(k+"\t"+inventory.get(k)+"\n");
        return sb.toString();
    }
    public void addItem( Item item , int ammount ){
        if( this.inventory.containsKey(item))
            this.inventory.put(item, this.inventory.get(item)+ammount);
        this.inventory.put(item,ammount);
    }
    public void addClient(Client c){
        this.clients.add(c);
    }
    public void addBill( Bill b ){
        this.bills.add(b);
    }
    public String getInventory() {
        StringBuilder sb = new StringBuilder();
        sb.append("Inventory (Alphabetical order)\n");
        sb.append("ID\tName\t\tStock\n");
        ArrayList<Item> items = new ArrayList<Item>(inventory.keySet());
        Collections.sort(items, new ByNameItemComparator());
        for( Item k : items)
            sb.append(k+"\t"+inventory.get(k)+"\n");
        return sb.toString();
    }
}
```

Código 6: Clase *Store* en el ejemplo de la Tienda

4. Problemas propuestos

Casi todos problemas de esta sección constan de un enunciado en el que se solicita el diseño e implementación de un programa en Java que tome como entrada ciertos datos leídos desde su entrada estándar y produzca como única salida los datos solicitados, que pueden ser cálculos o transformaciones hechas con los datos de entrada. En todos los problemas propuestos se debe asumir que las entradas cumplirán con lo estipulado en la sección "`Descripción de entrada`" de cada problema. Dicho esto, se debe entonces diseñar e implementar un programa que cumpla con lo estipulado en la sección "`Descripción de salida`", y que sea únicamente lo solicitado allí, nada más.

Teniendo en cuenta que en este documentos se tratan temas relacionados tanto con contenedores como con tipos de datos definidos por el usuario, se espera que en todas las soluciones se usen de alguna manera. Como requisito adicional, toda la lectura de datos (uso de *Scanner*) y toda impresión de datos a la salida estándar (uso de *System.out*) debe hacerse en el método *main*; es decir, en los demás métodos no puede haber ni lectura ni impresión de datos.

1. Centro de masas. Dados un conjunto de puntos con masa en el plano, hacer un programa que lea estos datos y calcule y muestre el centro de masas de estos puntos. En este problema se debe crear la clase *Point* y la clase *PointSet*. Vale la pena aclarar que, de manera estándar, Java tiene una clase Punto que se podráa usar aquí (Oracle, 1999b). El centro de masas está dado por la siguiente fórmula:

$$x_{cm} = \frac{\sum_{i=1}^{n} m_i * x_i}{\sum_{i=1}^{n} m_i}$$
$$y_{cm} = \frac{\sum_{i=1}^{n} m_i * y_i}{\sum_{i=1}^{n} m_i}$$

- Descripción de entrada: un número entero ***n*** seguido de ***n*** triplas de números. Cada tripla es un punto con su masa, primero la coordenada ***x***, luego la coordenada ***y*** y luego la masa ***m*** para ese punto.
- Descripción de salida: dos números, la coordenada ***x*** del centro de masa y la coordenada y del centro de masa de los puntos leídos.
- Ejemplo de entrada:
  5
  10 10 2
  10 20 4
  10 30 8
  10 40 1
  10 50 16
- Ejemplo de salida:
  10 38.065

1. Diseño de clase Triángulo. Diseñar una clase que sea la abstracción de un triángulo en el plano cartesiano.
2. Aquí se espera que los atributos de la clase triángulo sean tres puntos en el plano cartesiano. Adicional a los atributos, la clase debe tener comportamientos como: obtener lados, obtener área, obtener perímetro y obtener punto del centro de masas. Se recomienda usar la clase punto creada en el problema anterior. Diseñar también un método *main* y verificar el funcionamiento de la clase con la instanciación de varios ejemplares de la clase triángulo.
3. Diseño de la clase Círculo. Diseñar una clase que sea la abstracción de un círculo en el plano cartesiano. Esta clase debe tener dos atributos, un punto que será su centro, y un real que será su radio. Se recomienda usar la clase punto creada para problemas anteriores.
4. Se desea hacer un programa que tome como entrada los datos de estudiantes (nombres completos, números de documento) y una serie de notas, y produzca una lista Estudiante-Promedio ordenada de mejor a peor promedio de notas y muestra el resultado a manera de tabla con tres columnas: nombre, cédula y promedio. El formato del archivo de entrada es el siguiente:

La primera línea del archivo de entrada contiene un número entero positivo ***n***; este será la cantidad de líneas que siguen y cada línea representará la información de un estudiante. Cada línea tendrá el siguiente formato: nombre completo - número de documento - serie de notas entre 0 y 5, separadas por espacios.
Descripción de salida: la lista según lo solicitado.

- Ejemplo de entrada:
  4
  Rafael Alberto Velasquez-889993893-4 3 4 4.5 4 2 1
  Jose Figueroa-2334234-4 3 1 3 5
  Diego Castilla-232234-3 4 1 2 3 3 2 1 2.3 2.3
  Alberto Baron-234213-3 3 1
- Ejemplo de salida:
  Alberto Baron 234213 4.33333
  Rafael Alberto Velasquez 889993893 3.21429

5. Agregar al ejemplo de la tienda un método ***validPurshase*** en la clase *Store* que tome una factura y valide si se puede hacer la venta dado el inventario actual.
6. Agregar al ejemplo de la tienda un método ***makePurchase*** en el que se aplique la venta de una factura (Bill) restando los elementos vendidos al inventario actual.
7. Agregar al ejemplo de la tienda un atributo a la clase *Item* que tenga en cuenta las unidades en las que se venden los productos. El atributo debe llamarse *units*. Por ejemplo, para la leche podrían ser litros, para el arroz podrían ser kilos, etc. Hacer las modificaciones necesarias para que el ejemplo siga funcionando.
8. Agregar un método en la clase *Store* que muestre los nombres de los clientes en orden alfabético. Para ello, deben crear una clase comparadora de clientes por nombre, como se tiene para los *Items*.
9. Agregar un método a la clase *Store* que retorne una representación del inventario, organizado por la cantidad de elementos que hay en este, empezando por los ítems que más hay. Para esto, se debe crear una clase *ByAmmountItemCompartor*, la cual debe implementar la interfaz *Comparator* para organizar fácilmente los ítems respecto a la cantidad de elementos, de manera similar a como se hizo para el método ***getInventory***.
10. Cambiar el uso de ***TreeMap*** en la clase *Store* para usar *HashMap*. Para lograr hacer esto, se deben sobrescribir los métodos ***hashcode*** e ***equals*** en la clase *Item*. Analizar las diferencias en las impresiones de elementos. ¿Cambia el orden? ¿Por qué?
11. Cambiar los usos de ***TreeSet*** en la clase *Store*. Para ello, se deben sobrescribir los métodos ***hashcode*** e ***equals*** en las clases Bill y Client. Analizar las diferencias en las impresiones de elementos. ¿Cambia el orden? ¿Por qué?
12. Cambiar el uso de ***TreeMap*** por *HashMap* en la clase *Bill*. Con este fin, debe sobrescribir los métodos *hashcode* e *equals* en la clase *Item*. Analizar las diferencias en las impresiones de elementos. ¿Cambia el orden? ¿Por qué?
13. Diseñar e implementar una clase en Java que tenga los métodos necesarios para la generación de datos pseudoaleatorios que sirvan para probar los diferentes métodos y comportamientos de la clase.
14. Haciendo uso de la interfaz serializable y de la posibilidad de Java de almacenar instancias de objetos serializados en archivos, hacer persistir en disco duro la instancia de *store* para que permanezca la información, aún cuando se termine el programa. Seguido a esto, hacer un programa que tenga la capacidad tanto de guardar información serializada en disco como de cargar información de una tienda almacenada en disco.
15. Crear un formato de archivo plano para almacenar toda la información de la tienda. Hacer un programa que pueda leer esa información y llevarla a la memoria principal y hacer el proceso inverso, de memoria principal conducirla a un archivo de texto plano.

Referencias

Flórez, H. (2012). *Programación orientada a objetos usando Java*. Colombia: Ecoe Ediciones.

García, L. (2010). *Todo lo básico que debería saber: sobre programación orientada a objetos en Java*. Barranquilla, Colombia: Ediciones Uninorte; Grupo Editorial Ibáñez.

Hillar, G. (2015). *Learning Object-Oriented Programming*. Packt Publishing.

Jaramillo, S., Cardona, S. y Hernández, L. (2010). *Programación orientada a objetos*. Armenia, Colombia: Ediciones Elizcom.

López, J. y Gutiérrez, A. (2014). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*. México, D.F., México: Grupo Editorial Patria.

Oracle. (1999a). *Interfaces*. Recuperado de https://docs.oracle. com/javase/tutorial/java/landl/createinterface.html

Oracle. (1999b). *Class Point*. Recuperado de https://docs.oracle. com/javase/7/docs/api/java/awt/Point.html

Oviedo, E. (2015). *Lógica de programación orientada a objetos*. Colombia: Ecoe Ediciones, Universidad de Antioquia.

Sierra, K. y Bates, B. (2005). *Head First Java*, (2da Ed). Sebastopol, CA, USA: O'Reilly Media, Inc.

Vélez, J. , Peña, A. Gortázar, F. y Sánchez, A. (2014). *Diseñar y programar, todo es empezar: Una Introducción a la Programación Orientada a Objetos usando UML y Java*. Madrid, España: Universidad Rey Juan Carlos.

Weisfeld, M. (2008). *The Object-Oriented Thought Process*. Addison-Wesley Professional. Ediciones Elizcom.

INFORMACIÓN TÉCNICA

Ingeniería y Ciencias Básicas

Módulo: Conceptos Fundamentales de Programación
Unidad 4: Tipos y contenedores de datos
Escenario 8: Contenedores y tipos de datos definidos por el usuario

Autor: Rafael Niquefa

Asesor Pedagógico: Angie Laitón
Diseñador Gráfico: Rafael Niquefa
Asistente: Alejandra Morales

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.